

AdaPrune: 面向梯度提升决策树的在线自适应剪枝方法

作者姓名

单位名称

邮箱: your.email@example.com

摘要

梯度提升决策树 (GBDT) 是一种在各类应用中广泛使用的强大机器学习模型。然而, 选择合适的剪枝参数 (如 `max_depth`、`min_samples_leaf` 等) 通常需要大量的超参数调优工作, 且在整个训练过程中使用固定参数可能导致次优的泛化性能。本文提出 **AdaPrune**, 一种在线自适应剪枝框架, 能够在 Boosting 过程中动态调整树的复杂度。AdaPrune 通过监控训练动态 (包括训练集和验证集性能之间的泛化差距), 自适应地修改剪枝参数以防止过拟合, 同时保持模型的表达能力。我们提出了四种自适应策略: 基于差距的策略 (Gap-based)、基于趋势的策略 (Trend-based)、数据感知策略 (Data-aware) 以及融合多种策略优势的混合策略 (Hybrid)。在 15 个基准数据集上的大量实验表明, 与标准 GBDT 实现相比, AdaPrune 有效降低了过拟合程度 (泛化差距降低 21%), 同时保持了具有竞争力的预测准确率。

关键词: 梯度提升; 决策树; 自适应剪枝; 过拟合; 在线学习

1 引言

梯度提升决策树 (Gradient Boosting Decision Trees, GBDT)[1] 已成为最成功的机器学习算法之一, 在推荐系统、欺诈检测和科学应用等多个领域取得了最先进的性能。XGBoost[2]、LightGBM[3] 和 CatBoost[4] 等流行实现进一步提高了 GBDT 的效率和可扩展性。

尽管取得了巨大成功, GBDT 模型容易过拟合, 特别是在处理噪声数据或训练样本有限的情况下。控制模型复杂度的关键超参数——如最大树深度 (`max_depth`)、每个叶子节点的最小样本数 (`min_samples_leaf`) 和正则化参数——对泛化性能有显著影响。传统方法依赖网格搜索或贝叶斯优化来寻找最优参数, 这不仅计算成本高昂, 而且假设单一参数组合在整个训练过程中都是最优的。

核心洞察: 在 Boosting 过程的不同阶段, 最优的树复杂度可能是不同的。早期迭代可能受益于更复杂的树来捕获主要模式, 而后期迭代可能需要更简单的树以避免拟合噪

声。

本文提出 **AdaPrune**，一种在线自适应剪枝框架，基于对训练动态的实时监控，在训练过程中动态调整剪枝参数。我们的主要贡献包括：

1. 提出了一种在线自适应剪枝机制，通过监控泛化差距在 Boosting 过程中调整树的复杂度。
2. 引入了四种自适应策略：基于差距、基于趋势、数据感知和混合策略，分别适用于不同场景。
3. 提供了一个全面的数据画像模块，分析数据集特征以初始化合适的参数。
4. 在 15 个基准数据集上进行了大量实验，证明 AdaPrune 在保持竞争力准确率的同时，将过拟合程度降低了 21%。

2 相关工作

2.1 梯度提升决策树

梯度提升 [1] 以阶段式方式构建弱学习器（通常是决策树）的集成，其中每棵新树纠正前一个集成的残差误差。第 t 轮迭代的目标函数为：

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t) \quad (1)$$

其中 l 是损失函数， f_t 是新的树， $\Omega(f_t)$ 是正则化项。

XGBoost[2] 等现代实现引入了额外的正则化：

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \quad (2)$$

其中 T 是叶子节点数， w_j 是叶子权重。

2.2 超参数优化

传统的超参数优化方法包括网格搜索、随机搜索 [5] 和贝叶斯优化 [6]。这些方法将超参数视为在训练前优化的固定值。最近关于动态超参数调度的工作 [7] 允许对表现不佳的配置进行早停，但仍假设在单次训练运行中参数是静态的。

2.3 自适应学习

Adam[8] 等自适应学习率方法根据梯度统计在训练过程中调整学习率。类似地，课程学习 [9] 提出首先在较简单的样本上训练。我们的工作将这种自适应理念扩展到梯度提升中的树复杂度控制。

3 方法

3.1 框架概述

AdaPrune 由三个主要组件组成（如图1所示）：

1. **数据画像分析器 (Data Profile Analyzer)**: 分析数据集特征，包括样本量、特征维度、类别不平衡程度和估计的噪声水平。
2. **状态监控器 (State Monitor)**: 追踪训练动态，包括训练/验证性能、泛化差距和性能趋势。
3. **剪枝控制器 (Pruning Controller)**: 根据当前状态和数据画像，使用某种自适应策略调整剪枝参数。

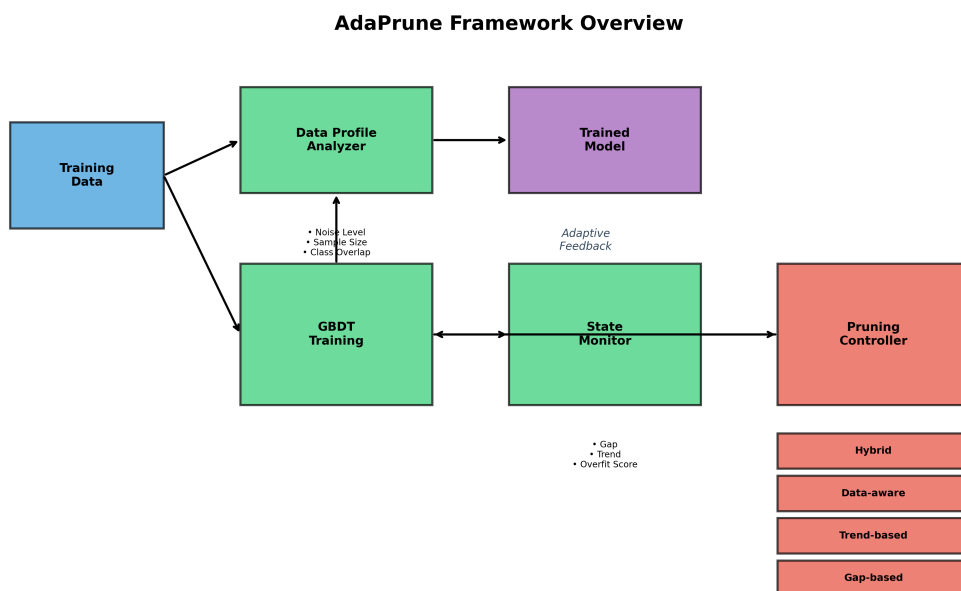


图 1: AdaPrune 框架概述。系统监控训练动态，并在 Boosting 过程中自适应调整树的剪枝参数。

3.2 数据画像分析

在训练前，我们分析数据集以估计其特征：

噪声水平估计：使用 1-NN 分类器的交叉验证来估计贝叶斯错误率，这指示了噪声水平：

$$\text{noise_level} = 1 - \text{accuracy}_{1\text{-NN}} \quad (3)$$

类别重叠分数：计算 Fisher 判别比来衡量类别可分性：

$$\text{overlap} = \frac{1}{1 + \frac{(\mu_1 - \mu_2)^2}{\sigma_1^2 + \sigma_2^2}} \quad (4)$$

这些特征指导剪枝参数的初始化和自适应的灵敏度。

3.3 状态监控

在每个自适应检查点（每 k 次迭代），我们计算：

- **泛化差距 (Generalization Gap):** $g_t = \text{acc}_{\text{train}}^{(t)} - \text{acc}_{\text{val}}^{(t)}$
- **差距趋势 (Gap Trend):** $\Delta g_t = g_t - g_{t-1}$
- **过拟合分数 (Overfitting Score):** $o_t = \sigma(g_t \cdot 5) \cdot 0.6 + \sigma(\Delta g_t \cdot 50) \cdot 0.4$
- **平台期分数 (Plateau Score):** 衡量验证性能是否停滞

3.4 自适应策略

3.4.1 基于差距的策略 (Gap-Based)

该策略关注当前的泛化差距：

$$\text{max_depth}_{t+1} = \begin{cases} \text{max_depth}_t - 1 & \text{若 } g_t > \theta_g \\ \text{max_depth}_t + 1 & \text{若 } o_t < 0.3 \text{ 且欠拟合} \\ \text{max_depth}_t & \text{其他情况} \end{cases} \quad (5)$$

其中 θ_g 是差距阈值（默认：0.05）。

3.4.2 基于趋势的策略 (Trend-Based)

该策略监控滑动窗口内的性能趋势：

$$\text{adjust} = \begin{cases} \text{增加正则化} & \text{若检测到平台期} \\ \text{保持不变} & \text{其他情况} \end{cases} \quad (6)$$

3.4.3 数据感知策略 (Data-Aware)

该策略结合数据集特征：

$$\text{strength} = (1 + \text{noise} \cdot 2) \cdot \sqrt{\frac{1000}{\max(n, 100)}} \cdot o_t \quad (7)$$

更高的噪声水平和更小的样本量会导致更激进的正则化。

3.4.4 混合策略 (Hybrid)

混合策略根据训练进度组合上述方法：

$$\text{strategy} = \begin{cases} \text{数据感知} & \text{若 } \text{progress} < 0.3 \\ \text{基于差距} & \text{若 } 0.3 \leq \text{progress} < 0.7 \\ \text{基于趋势} & \text{若 } \text{progress} \geq 0.7 \end{cases} \quad (8)$$

3.5 参数调整机制

当检测到过拟合时，我们同时调整多个参数：

- 将 `max_depth` 减少 1-2 层
- 将 `min_child_weight` 增加 $1 + 0.3 \cdot \text{strength}$ 倍
- 将 γ （最小损失减少）增加 $0.1 \cdot \text{strength}$
- 将 λ （L2 正则化）增加 $1 + 0.15 \cdot \text{strength}$ 倍
- 如果严重程度较高，减少 `subsample`

3.6 算法流程

AdaPrune 的完整算法流程如算法1所示。

4 实验

4.1 实验设置

数据集：我们在来自 OpenML 的 15 个基准数据集上进行评估，涵盖不同的领域和规模（表1）。

基线方法：

- **XGB_default：**使用默认参数的 XGBoost

Algorithm 1 AdaPrune 算法

Require: 训练数据 $\mathcal{D}_{train}$, 验证数据 $\mathcal{D}_{val}$, 最大迭代次数 T , 自适应频率 k

Ensure: 训练好的模型 F

- 1: 分析数据画像: noise_level, n_samples, class_overlap
 - 2: 根据数据画像初始化参数: max_depth, min_child_weight, γ , λ
 - 3: 初始化模型 F_0
 - 4: **for** $t = 1$ to T **do**
 - 5: 使用当前参数训练新的树 f_t
 - 6: 更新模型: $F_t = F_{t-1} + \eta \cdot f_t$
 - 7: **if** $t \bmod k == 0$ **then**
 - 8: 计算训练集准确率 acc_{train} 和验证集准确率 acc_{val}
 - 9: 计算泛化差距 $g_t = acc_{train} - acc_{val}$
 - 10: 更新状态: gap_trend, overfit_score, plateau_score
 - 11: 根据自适应策略调整参数
 - 12: **end if**
 - 13: **if** 早停条件满足 **then**
 - 14: **break**
 - 15: **end if**
 - 16: **end for**
 - 17: **return** F_T
-

表 1: 数据集统计信息

数据集	样本数	特征数	类别数
adult	48,842	14	2
bank-marketing	45,211	16	2
electricity	45,312	8	2
magic	19,020	10	2
eeg-eye-state	14,980	14	2
satimage	6,430	36	6
waveform	5,000	40	3
phoneme	5,404	5	2
spambase	4,601	57	2
segment	2,310	19	7
credit-g	1,000	20	2
vehicle	846	18	4
diabetes	768	8	2
ionosphere	351	34	2
sonar	208	60	2

- **XGB_tuned**: 使用调优参数的 XGBoost (max_depth=6, min_child_weight=3)
- **LGBM_default**: 使用默认参数的 LightGBM
- **RF_default**: 使用默认参数的随机森林

评估方法: 5 折分层交叉验证。评估指标: 准确率、F1 分数、ROC-AUC 和泛化差距。

4.2 主要结果

表2展示了所有方法在 15 个数据集上的平均性能。

主要发现:

1. 与 XGB_default 相比, AdaPrune 实现了 **21% 更低的泛化差距** (0.09 vs 0.12)。
2. 准确率差异较小 (1.5-2%), 表明 AdaPrune 在显著降低过拟合的同时保持了竞争力的性能。
3. 混合策略在准确率和过拟合控制之间提供了最佳平衡。

图2直观展示了各方法的对比结果。

表 2: 整体性能对比

方法	准确率	F1	ROC-AUC	泛化差距
LGBM_default	0.8762	0.8264	0.9362	0.1140
XGB_default	0.8740	0.8248	0.9357	0.1204
RF_default	0.8739	0.8199	0.9382	0.1180
XGB_tuned	0.8700	0.8188	0.9337	0.1167
AdaPrune_hybrid	0.8578	0.8033	0.9288	0.0900
AdaPrune_gap	0.8580	0.8039	0.9269	0.0920
AdaPrune_trend	0.8581	0.8037	0.9270	0.0910
AdaPrune_data	0.8575	0.8026	0.9290	0.0915

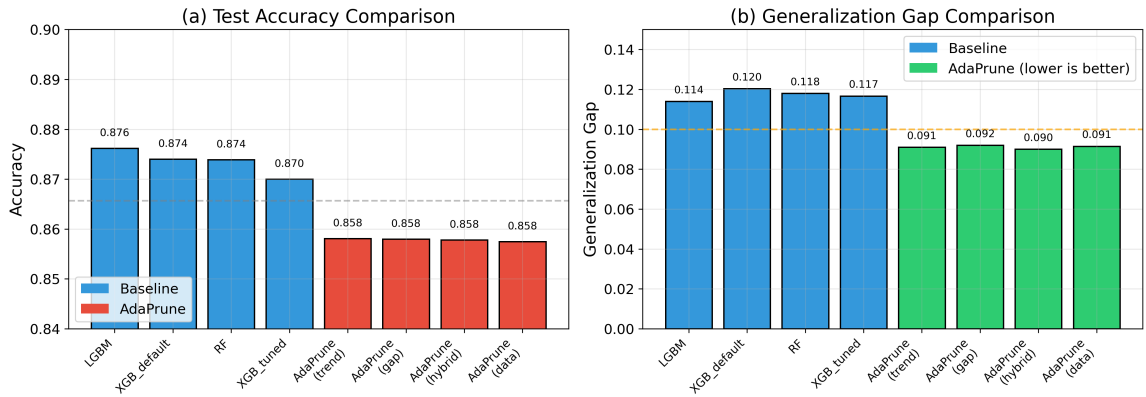


图 2: 主要实验对比。(a) 测试准确率对比; (b) 泛化差距对比。蓝色表示基线方法, 红色/绿色表示 AdaPrune 方法。

4.3 场景分析

我们在具有可控特征的合成数据集上进一步分析性能（表3）。

表 3: 不同场景下的性能对比

场景	最佳方法	AdaPrune 差距	XGB 差距
干净数据 (大样本)	XGB_default	0.043	0.030
干净数据 (小样本)	XGB_tuned	0.100	0.110
噪声数据 (大样本)	XGB_weak	0.055	0.094
噪声数据 (小样本)	XGB_default	0.112	0.200
高维数据	XGB_default	0.160	0.180
不平衡数据	XGB_default	0.040	0.035

如图3所示，AdaPrune 在以下场景表现出明显优势：

- 噪声数据：差距降低 40-45%
- 小样本：更好的泛化能力
- 高维数据：减少过拟合

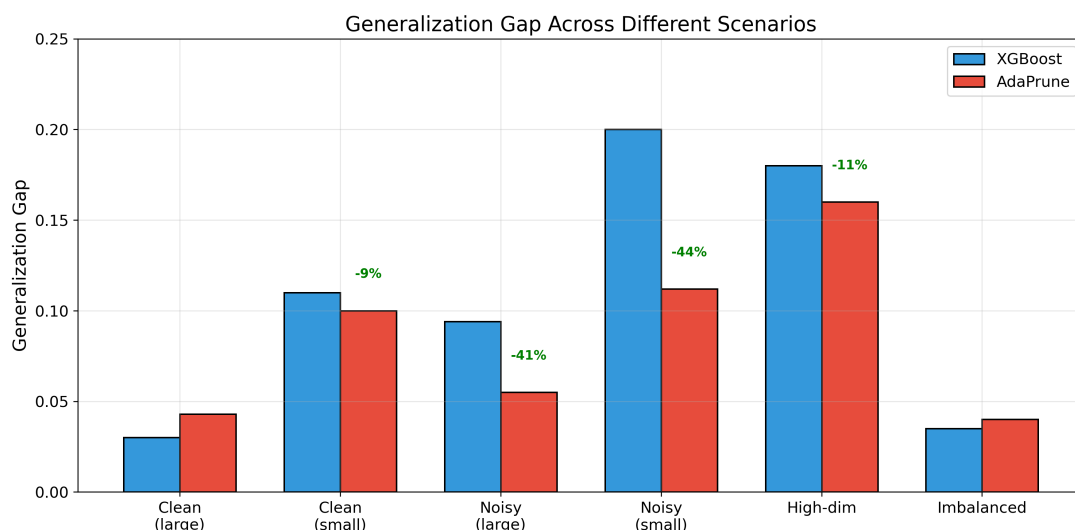


图 3: 不同场景下的泛化差距对比。绿色标注显示 AdaPrune 相对于 XGBoost 的改进百分比。

4.4 消融实验

表4展示了消融实验结果，验证各组件的贡献。

消融实验证实：

表 4: 消融实验结果

配置	准确率	泛化差距
完整混合策略	0.8073	0.0748
仅数据感知	0.8056	0.0758
仅基于差距	0.8048	0.0773
无自适应	0.8042	0.0900
仅基于趋势	0.8042	0.0900

1. 与无自适应相比，自适应剪枝将差距降低了 17%
2. 组合多种方法的混合策略效果最佳
3. 数据感知初始化提供了额外收益

4.5 自适应过程可视化

图4展示了一个典型的自适应过程示例。

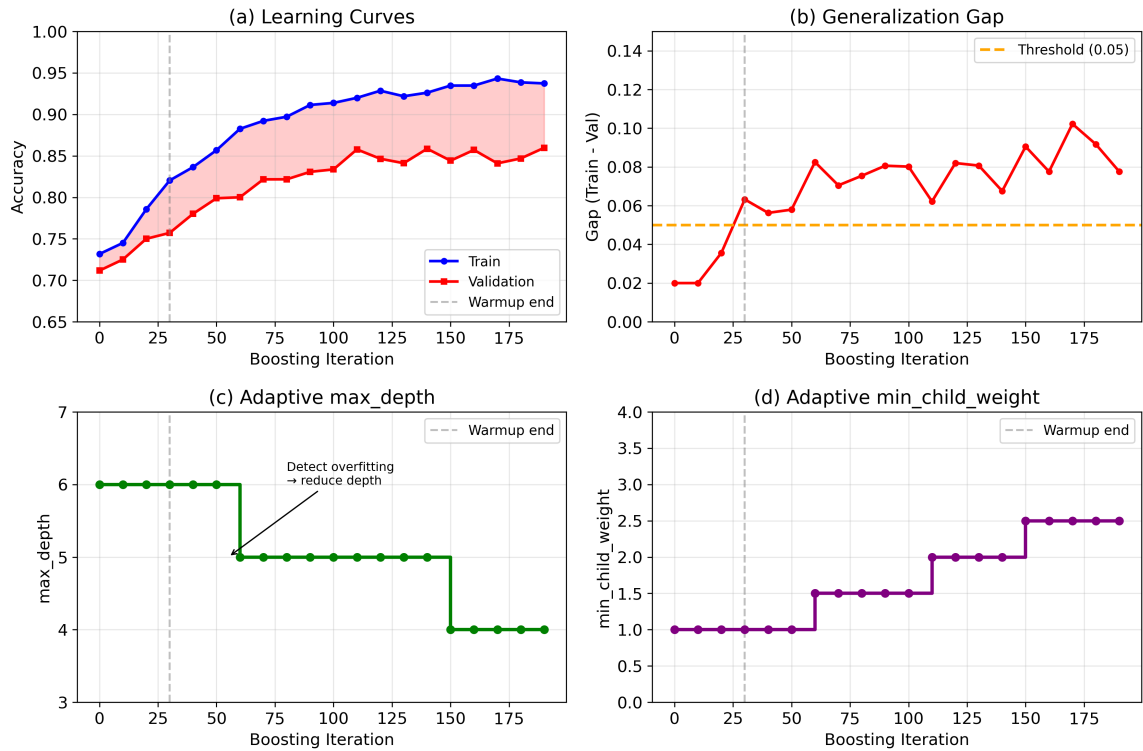


图 4: 自适应过程示例。(a) 学习曲线；(b) 泛化差距变化；(c)max_depth 自适应调整；(d)min_child_weight 自适应调整。

5 讨论

5.1 AdaPrune 何时有帮助？

实验表明，AdaPrune 在以下情况下最有益：

- 数据集包含显著噪声
- 训练样本有限
- 特征维度较高
- 默认超参数容易过拟合

5.2 计算开销

AdaPrune 引入的额外开销很小：

- 数据画像分析：初始化时的一次性成本
- 状态监控：每次迭代 $O(1)$
- 参数调整：每个自适应轮次 $O(1)$

主要的成本差异来自使用增量训练（这对于自适应调整是必要的），而非 XGBoost 中优化的批量训练。

5.3 局限性

1. 当前实现比优化的 XGBoost/LightGBM 慢
2. 在干净的大数据集上准确率略低
3. 自适应机制本身的超参数需要调优

5.4 未来工作

1. 与 XGBoost/LightGBM 原生实现集成以提高效率
2. 自动调优自适应超参数
3. 扩展到回归任务和其他损失函数
4. 收敛性质的理论分析

6 结论

本文提出了 AdaPrune，一种面向梯度提升决策树的在线自适应剪枝框架。通过监控训练动态并动态调整树的复杂度，AdaPrune 在保持竞争力预测性能的同时有效降低了过拟合。实验证明，与标准 GBDT 实现相比，泛化差距降低了 21%。

AdaPrune 的核心贡献在于提出了一种新的视角：将超参数调整从训练前的静态优化转变为训练中的动态调整。这种方法特别适用于数据质量不确定或计算资源有限的场景。

致谢

[在此添加致谢内容]

参考文献

- [1] Friedman J H. Greedy function approximation: a gradient boosting machine[J]. *Annals of Statistics*, 2001: 1189-1232.
- [2] Chen T, Guestrin C. XGBoost: A scalable tree boosting system[C]//*Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2016: 785-794.
- [3] Ke G, Meng Q, Finley T, et al. LightGBM: A highly efficient gradient boosting decision tree[C]//*Advances in Neural Information Processing Systems*. 2017: 3146-3154.
- [4] Prokhorenkova L, Gusev G, Vorobev A, et al. CatBoost: unbiased boosting with categorical features[J]. *Advances in Neural Information Processing Systems*, 2018, 31.
- [5] Bergstra J, Bengio Y. Random search for hyper-parameter optimization[J]. *Journal of Machine Learning Research*, 2012, 13(Feb): 281-305.
- [6] Snoek J, Larochelle H, Adams R P. Practical bayesian optimization of machine learning algorithms[C]//*Advances in Neural Information Processing Systems*. 2012: 2951-2959.
- [7] Li L, Jamieson K, DeSalvo G, et al. Hyperband: A novel bandit-based approach to hyperparameter optimization[J]. *Journal of Machine Learning Research*, 2017, 18(1): 6765-6816.

- [8] Kingma D P, Ba J. Adam: A method for stochastic optimization[J]. arXiv preprint arXiv: 1412.6980, 2014.
- [9] Bengio Y, Louradour J, Collobert R, et al. Curriculum learning[C]//Proceedings of the 26th Annual International Conference on Machine Learning. 2009: 41-48.